Do Code LLMs Understand Design Patterns?

Zhenyu Pan
Northwestern University
zhenyupan@u.northwestern.edu

Xuefeng Song
Northwestern University
xuefengsong2026@u.northwestern.edu

Yunkun Wang
Zhejiang University
wangykun@zju.edu.cn

Rongyu Cao
Alibaba Group
caorongyu.cry@alibaba-inc.com

Binhua Li
Alibaba Group
binhua.lbh@alibaba-inc.com

Yongbin Li
Alibaba Group
shuide.lyb@alibaba-inc.com

Han Liu
Northwestern University
hanliu@northwestern.edu

2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code) | 979-8-3315-2615-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/LLM4Code66737.2025.00031

***Abstract*—Code Large Language Models (LLMs) demonstrate great versatility in adapting to various downstream tasks, including code generation and completion, as well as bug detection and fixing. However, Code LLMs often fail to capture existing coding standards, leading to the generation of code that conflicts with the required design patterns for a given project. As a result, developers must post-process to adapt the generated code to the project's design norms. In this work, we empirically investigate the biases of Code LLMs in software development. Through carefully designed experiments, we assess the models' understanding of design patterns across recognition, comprehension, and generation. Our findings reveal that biases in Code LLMs significantly affect the reliability of downstream tasks.**

I. Introduction

LLMs transform traditional paradigms across various fields [6]. Among these [7], Code LLMs, trained on extensive code data, demonstrate great versatility in code-related tasks. They facilitate automated software development through downstream tasks such as code generation and completion, bug localization and repair, and security detection and enhancement, which significantly improves developer productivity.

However, there are still considerable challenges in applying code LLMs in real-world scenarios, especially regarding specific aspects of software engineering. For instance, in complex repository environments, it is difficult to extract the necessary background knowledge from a large amount of information and follow project-specific conventions during code generation. Current code LLMs often fail to properly understand the existing design patterns and coding styles of a project, leading to generated code that does not meet project requirements. This mismatch creates an additional burden for developers, such as requiring extensive code revisions and post-processing.

Recent empirical studies primarily evaluate code LLMs on tasks like code generation, bug detection, and code summarization, often using benchmarks such as CodeXGLUE [10]. While these analyses have provided insights into generation accuracy and contextual understanding, the role of design patterns in software engineering has been largely overlooked. Design patterns are essential for maintaining software quality, particularly in object-oriented development. Emphasizing bias analysis is crucial, especially regarding potential biases in code LLMs under different design patterns, as such biases may impact the reliability of downstream tasks.

To bridge this gap, we propose an empirical study to evaluate code LLMs' understanding and generation of design patterns, focusing on whether these models can classify and generate code while adhering to established patterns like Singleton and Factory. For our dataset, we manually selected high-quality repositories from GitHub for both Python and Java, each corresponding to 12 design patterns, with two repositories chosen for each pattern in each language. Based on this, we designed three experiments to assess the capabilities of Code LLMs in recognition, generation, and comprehension of design patterns. For recognition, we conducted a Design Pattern Classification experiment to classify the design patterns in a given code file. For generation and comprehension, we designed two sets of experiments involving code completion and function generation, both with and without prior information about the design pattern. Based on the evaluation of these three capabilities, we provide an analysis of the strengths and weaknesses of Code LLMs in handling design patterns.

The main contributions of our work are : (1) We propose an evaluation framework for recognizing, generating, and understanding design patterns, (2) Through experiments, we demonstrate that code LLMs exhibit biases when adhering to specific design patterns. (3) We illustrate how these biases affect the reliability of code generation and the additional workload imposed on developers. (4) We emphasize the significance of our work for future research and practical applications, such as improving model training and guiding models to better follow development standards.

II. Related Work

A. Code LLMs

Code LLMs demonstrate notable advancements, each with distinct strengths and limitations. The GPT-4 series excel in reasoning, multi-modal tasks, and code generation, but requires high computational resources and shows variability across updates [1]. Claude 3.5 focuses on safety and ethics in code generation, performing well in simple tasks but struggling with complex logic [2]. CodeQwen is efficient in domain-specific tasks like structured retrieval and technical analysis but lacks adaptability for general use [3]. The LLaMA 3.1 series are open-source, emphasizing efficiency and accessibility, but their reliance on public datasets limits generalization in niche

979-8-3315-2615-3/25/$31.00 ©2025 IEEE
DOI 10.1109/LLM4Code66737.2025.00031

scenarios [4]. Mistral is optimized for lightweight tasks and excels in long-context scenarios with its 128k context window, though it lacks multi-modal capabilities [5].

B. Empirical Studies on Code LLMs

Recent empirical studies on code LLMs focus on evaluating code generation, bug detection, code summarization, and automated documentation [17]. Researchers explore factors such as model architecture variations, the diversity and size of training datasets to understand their impact on the code LLMs in handling diverse code tasks [13]. Additionally, they investigate the model ability to comprehend contextual information, maintain code consistency, and manage complex programming constructs [14]. Despite these analyses, the role of design patterns in software engineering remains under-explored. Design patterns are crucial in software development, particularly in object-oriented programming, as they enhance the overall quality and maintainability of projects. However, there is no analysis of code LLMs' abilities to work with design patterns. To address this gap, we propose a focused study on code LLMs' understanding of design patterns. Specifically, we aim to assess whether code LLMs can accurately classify code files based on established design patterns and evaluate their ability to complete code while preserving patterns. By measuring the similarity between generated and original code, we seeks to provide deeper insights into the capabilities of code LLMs in applying software design patterns.

III. Design Pattern Evaluation

A. Experiment design

1) Overview: We evaluate the design pattern comprehension ability of code LLMs in different tasks across two programming languages, Python and Java: (i) **Design Pattern Classification**, which assesses the model's ability to identify and classify different object-oriented design patterns, such as Singleton, Factory, and Observer, within given code snippets. The objective is to determine if the model can accurately recognize structural and behavioral characteristics that define each design pattern in both Python and Java; (ii) **Line Completion**, where the model completes missing lines within a code snippet that follows a particular design pattern. This task tests the model's understanding of the context and requirements of specific design patterns, as it must generate lines that adhere to the pattern's conventions in each language; and (iii) **Function Generation**, which evaluates the model's ability to generate entire functions with given description in with/without given design pattern, assessing whether the model can recognize and apply structural and procedural elements of the pattern to fulfill functional requirements effectively in both Python and Java.

2) Models: Our study draws upon a wide range of state-of-the-art large language models (LLMs), each selected to showcase distinct strengths across various model families and architectural designs. We employ general-purpose models, such as GPT-4 and Claude, alongside code-focused models like GLM and DeepSeek, which are tailored for tasks involving code understanding and generation. Additionally, we include models from specific series, such as Qwen and LLaMA, and incorporate multimodal and multilingual capabilities through options like Yi and Mistral. This diverse model set enables a robust evaluation of LLM performance on code-centric tasks, with a particular focus on assessing capabilities in design pattern comprehension within domain-specialized applications.

3) Datasets: We manually selected 48 high-quality repositories from GitHub, with 24 in Python and 24 in Java, representing 12 design patterns. Each pattern was represented by two repositories per language, and the code files were categorized into three levels based on complexity: simple, moderate, and difficult. For the Design Pattern Classification task, we provided code files to LLMs to determine the design pattern. In the Line Completion task, we randomly removed three separate lines of effective code and asked the LLMs to complete it based on the context. For the Function Generation task, we extracted a function, used GPT-4 to write a description, and provided the context and description to the LLMs to generate the function. We test both with and without given design pattern to evaluate the models' understanding.

4) Metrics: For the classification results, we used simple accuracy to evaluate the models' performance. For the code completion and generation tasks, we used two metrics: code similarity (CS) and code edit similarity (ES). CS measures the overlap between the generated code and the reference code. We use difflib's SequenceMatcher, which employs the Ratcliff/Obershelp algorithm to finds the Longest Matching Subsequence (LCS) between two sequences and then recursively processes the unmatched parts to calculate the similarity ratio. ES measures the minimal number of edit operations (insertion, deletion, substitution) required to transform the generated code into the reference code, normalized by the length of the reference.

B. Evaluation on Design Pattern Classification

1) Empirical Analysis: As shown in Table I, GPT-4o and Llama-31-70B achieved the highest overall accuracy, both scoring 38.81% across all datasets. Both GPT-4o and Llama-31-70B achieved the highest accuracy on Java Easy at 71.43%. For the Python datasets, GPT-4o led with an overall accuracy of 29.73% on Python All. On Python Easy, several models, including CodeQwen, Mistral-123B, Yi-34B, and GLM-3-6B, matched high accuracy levels at 44.44%. A general trend across all models is a decrease in performance from Java to Python and from Easy to Hard datasets. This suggests that the LLMs find Python code and more complex code samples more challenging, possibly due to differences in language syntax or less exposure to complex patterns during training.

From the heat map shown in Figure 1, when using LLMs to identify design patterns in code, Singleton and Factory patterns are often over-predicted or misattributed due to their prevalence and recognizable structure. Singleton involves centralized object management, which overlaps with patterns like Facade, Proxy, or Command, leading to frequent misclassification. Factory, similarly, is confused with Abstract Factory,

TABLE I
RESULTS OF DESIGN PATTERN CLASSIFICATION BY DIFFERENT LLMS ON JAVA AND PYTHON CODE FILES OF VARYING COMPLEXITY

Dataset	OpenAI Models		Qwen Models		Llama Models		Other Models				
	GPT-4	GPT-4o	CodeQwen	Qwen_25_72B	Llama_31_405B	Llama_31_70B	DeepSeek_V2	Claude35	Mistral_123B	Yi_34B	GLM_3_6B
Java Easy	64.29	71.43	42.86	71.43	64.29	71.43	64.29	42.86	57.14	57.14	57.14
Java Medium	None	14.29	None	28.57	28.57	14.29	None	14.29	None	14.29	14.29
Java Hard	33.33	44.44	22.22	22.22	33.33	55.56	33.33	44.44	44.44	33.33	11.11
Python Easy	33.33	44.44	44.44	33.33	33.33	33.33	22.22	11.11	44.44	44.44	44.44
Python Medium	35.71	42.86	42.86	35.71	35.71	28.57	35.71	35.71	35.71	28.57	21.43
Python Hard	None	7.14	None	7.14	14.29	21.43	7.14	14.29	7.14	14.29	14.29

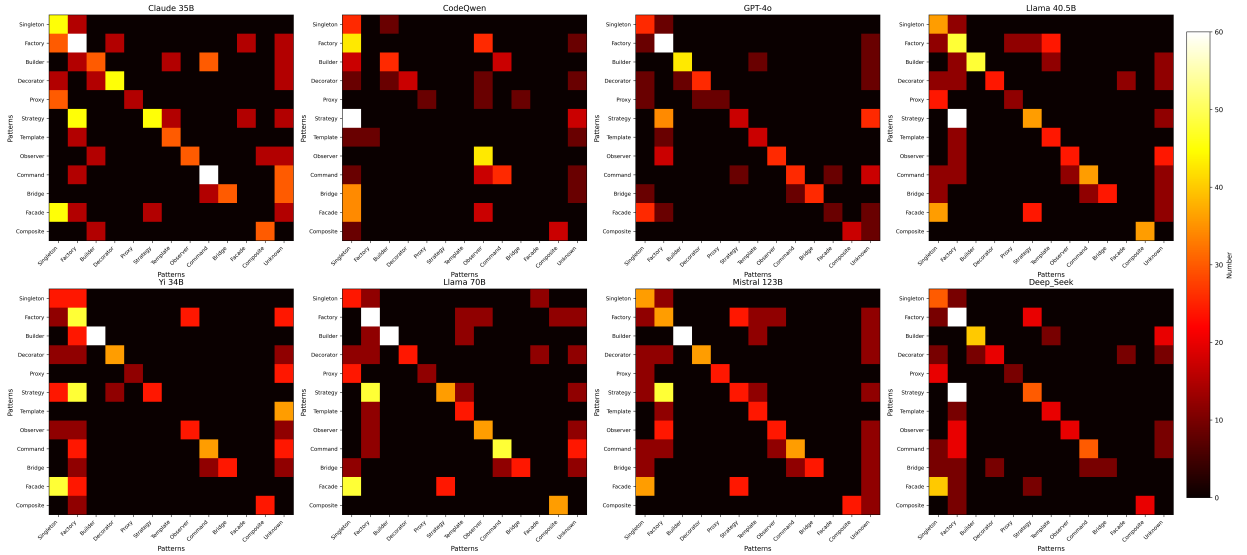

Fig. 1. Heatmaps of misclassifications by 8 LLMs across 12 design patterns involved in the experiments.

Builder, or Strategy due to shared object creation logic. Frameworks like Spring (Java) or Python's dependency injection further blur these distinctions, making it hard for LLMs to discern intent. Real-world implementations often combine patterns, adding to the challenge. LLMs heuristically rely on static methods, single-instance management, or object creation cues, which can align with Singleton or Factory but are not exclusive to them. Ambiguity in natural language prompts and insufficient training data also contribute to frequent misclassifications. Addressing these issues requires improved training data, clearer distinctions, and refined contextual understanding of overlapping features in code. LLMs also struggle with the Facade pattern due to its abstract nature and contextual dependency. Unlike Singleton or Factory, which have distinct structural markers, Facade simplifies access to complex subsystems via a unified interface, making its identification challenging. Its reliance on naming conventions and subtle structural cues, along with integration with other dominant patterns, often masks its intent. This complexity, combined with insufficient training examples, makes Facade one of the harder patterns for LLMs to reliably identify.

Insights: The classification results indicate that even the best-performing models, GPT-4o and Llama-31-70B, achieved only 38.81% overall accuracy, highlighting substantial room for improvement in design pattern recognition. A consistent decline in performance from Java to Python and from Easy to Hard datasets suggests that LLMs find Python code and complex samples more challenging, due to syntax differences and less exposure during training. The heatmaps reveal that Singleton and Factory patterns are often over-predicted or misclassified. Their prevalence and distinctive structures make them default predictions, but overlaps with patterns like Facade or Strategy lead to confusion. LLMs particularly struggle with the Facade pattern due to its abstract nature and lack of explicit structural markers, making it difficult to identify without contextual understanding. These findings underscore the need for enhanced training data, clearer distinctions between patterns, and improved contextual comprehension within LLMs to increase reliability in software development tasks.

C. Evaluation on Line Completion and Function Generation

1) Empirical Analysis: As shown in Table II, we find: (1) CS and ES Relationship: Higher CS correlates with higher ES, meaning that code closely matching the original typically requires less effort to adapt. However, this relationship isn't always linear. For instance, GPT-4o achieves high CS without design pattern knowledge, but its ES scores suggest significant edits may still be required, especially in line completion tasks. (2) Impact of Design Pattern Knowledge and Model Variability: Several models, including GPT-4o and Llama-31-405B, perform better without design pattern information, implying reliance on internalized patterns from training. Claude35 consistently excels in generating accurate, modifiable code, while models like CodeQwen show lower performance. Function generation often yields higher similarity scores than line completion, suggesting better model handling of larger code segments. Overall, improving design pattern integration, practical utility, and task-specific optimization could enhance LLM reliability and reduce manual editing efforts.

TABLE II

OVERALL COMPARISON OF DIFFERENT LLMS ON LINE COMPLETION AND FUNCTION GENERATION WITH AND WITHOUT PRIOR KNOWLEDGE OF DESIGN PATTERN. EACH RESULT BOX SHOWS LINE COMPLETION ON THE LEFT AND FUNCTION GENERATION ON THE RIGHT, SEPARATED BY A '/'.

Models	OpenAI Models				Qwen Models				Llama Models				Other Models									
	GPT-4		GPT-4o		CodeQwen		Qwen_25_72B		Llama_31_405B		Llama_31_70B		DeepSeek_V2		Claude35		Mistral_123B		Yi_34B		GLM_3_6B	
Design Pattern Provided (Y/N)	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No
Code Similarity (%)	24.68/23.23	22.99/31.05	37.35/32.93	51.31/55.71	5.4/11.20	5.50/10.79	20.27/39.67	17.84/42.75	37.02/28.08	41.02/44.70	26.59/29.67	26.49/41.55	18.01/19.89	12.11/30.90	45.75/32.45	48.16/33.63	29.83/10.02	40.16/39.58	8.17/14.65	9.30/18.20	7.99/10.65	8.61/10.21
Edit Similarity (%)	23.91/26.08	21.21/32.08	33.30/32.77	43.72/54.27	6.2/12.21	5.54/13.74	20/39.11	17.72/42.34	32.58/25.75	34.78/45.73	24.36/28.97	24.98/43.21	18.48/21.01	13.54/31.41	37.35/29.91	41.50/33.40	28.31/16.51	35.22/38.62	8.90/19.17	9.96/23.99	11.39/15.60	12.60/16.06

TABLE III

COMPARISON OF LLMS ON FUNCTION GENERATION FOR VARIOUS DESIGN PATTERNS WITH AND WITHOUT PRIOR KNOWLEDGE OF THE CODE FILE'S DESIGN PATTERN. EACH RESULT BOX SHOWS CODE SIMILARITY (%) ON THE LEFT AND EDIT SIMILARITY (%) ON THE RIGHT, SEPARATED BY A '/'.

Models	OpenAI Models				Qwen Models				Llama Models				Other Models									
	GPT-4		GPT-4o		CodeQwen		Qwen_25_72B		Llama_31_405B		Llama_31_70B		DeepSeek_V2		Claude35		Mistral_123B		Yi_34B		GLM_3_6B	
Design Pattern Provided (Y/N)	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No
Singleton	33.64/39.89	13.20/32.10	36.15/32.77	42.73/50.79	2.74/5.47	2.48/4.98	52.57/44.11	35.19/36.03	35.08/33.22	32.52/58.02	37.37/34.83	39.60/49.90	18.55/14.65	19.77/19.78	22.82/21.85	12.91/33.17	6.63/11.40	62.12/55.73	23.50/21.19	13.78/22.92	19.07/22.12	7.94/15.22
Factory	34.31/34.23	34.72/30.64	20.22/23.34	47.92/52.73	14.57/15.52	8.99/11.65	25.82/31.21	31.50/29.85	32.55/32.66	58.14/54.32	29.24/28.42	36.80/34.45	14.76/20.60	23.06/25.77	32.47/33.00	15.53/19.52	7.72/17.70	25.30/26.07	6.55/15.98	8.92/12.92	6.55/15.16	2.69/8.58
Builder	6.68/21.07	13.41/26.04	28.12/41.23	33.89/43.64	14.41/16.52	15.13/20.81	10.87/21.00	9.31/17.46	15.90/24.81	34.07/40.58	6.39/19.64	13.94/22.96	3.22/11.00	9.47/15.80	11.70/24.41	17.47/27.47	8.57/17.39	12.77/17.98	9.76/25.77	10.91/30.16	5.13/15.32	4.95/13.52
Decorator	31.44/35.37	41.15/31.48	33.10/32.20	62.03/62.57	17.75/24.17	13.24/23.33	41.55/36.47	61.06/61.04	32.67/24.83	52.00/49.47	32.01/32.19	53.69/52.46	30.04/36.05	52.81/52.24	36.84/33.21	49.56/53.39	13.81/20.76	37.55/35.70	19.45/26.41	25.59/33.44	24.10/23.82	30.20/27.96
Proxy	35.48/34.06	27.18/20.18	36.90/37.00	36.22/32.08	27.69/28.86	9.77/14.50	36.48/35.62	36.70/32.98	29.37/27.22	35.16/30.65	28.38/24.52	35.08/30.62	21.08/15.53	33.44/31.18	26.94/23.56	26.23/19.31	5.72/14.46	34.03/29.30	4.71/15.56	6.01/4.95	12.99/15.49	2.82/12.13
Strategy	29.60/28.45	43.32/43.18	37.28/35.75	65.08/60.22	7.39/8.44	13.76/11.65	37.91/37.32	48.61/47.48	25.03/23.83	44.20/44.36	20.86/21.84	44.03/46.95	16.98/20.03	34.95/34.12	35.59/31.12	41.57/38.08	10.28/17.36	54.15/50.15	19.65/19.64	19.22/21.19	10.26/15.66	18.51/23.00
Template	33.62/33.33	54.19/46.37	62.20/51.98	65.59/60.11	10.49/7.06	27.37/21.76	66.17/61.46	62.40/57.79	49.88/37.91	57.13/50.01	44.44/36.74	66.45/66.09	30.41/26.21	36.75/35.58	59.31/43.98	48.59/38.56	16.69/18.04	63.12/50.14	2.51/6.36	3.09/7.10	8.50/14.86	1.95/11.44
Observer	7.67/5.86	5.08/5.30	15.46/10.29	76.48/70.35	7.96/6.58	6.73/4.92	48.23/44.82	77.50/64.55	13.79/10.30	70.38/62.31	28.48/20.16	43.07/40.17	16.23/11.56	43.73/29.23	45.53/29.67	36.48/23.06	12.21/9.09	48.02/45.74	42.08/36.55	41.13/34.52	14.19/9.68	9.02/7.12
Command	16.30/16.03	31.19/36.99	24.80/19.02	51.71/52.11	7.55/6.00	1.65/9.98	64.51/55.23	39.66/46.10	24.07/14.96	35.94/45.65	49.09/37.33	36.70/42.28	24.88/18.05	37.82/45.08	27.24/18.61	49.60/40.90	11.08/13.43	37.39/44.05	12.67/10.79	32.66/38.14	3.68/12.40	8.59/15.18
Bridge	6.78/17.21	6.67/15.98	37.07/40.71	37.58/40.12	11.55/11.01	12.05/12.66	37.14/39.17	30.15/28.63	28.94/27.63	20.12/20.08	31.08/36.29	21.33/18.23	32.71/30.17	25.15/24.73	28.14/28.91	26.54/22.94	3.02/16.23	32.19/32.84	8.46/21.26	10.85/18.09	5.15/7.99	4.03/7.24
Facade	11.29/11.67	39.81/41.41	32.54/31.91	56.92/56.33	5.22/5.12	3.71/5.62	29.16/31.38	39.27/39.57	30.78/25.05	53.04/50.49	37.99/38.90	59.49/61.47	15.44/21.76	19.06/23.43	36.57/34.69	41.95/37.29	7.76/13.57	29.14/32.45	15.13/14.72	18.29/24.81	8.01/12.65	3.98/15.25

Table III highlights that providing design pattern improves model performance, as seen with patterns like Singleton, where GPT-4 achieved higher Code Similarity (33.64%) with pattern knowledge than without (13.20%). However, exceptions exist; some models, such as GPT-4o, performed better without design pattern knowledge in patterns like Observer, suggesting effective reliance on internalized pattern knowledge. Performance varied across patterns, with Template and Decorator yielding higher scores, while Builder, Bridge, and Facade proved more challenging. A nuanced relationship between CS and ES was observed. Higher CS often corresponds with higher ES, suggesting that code closer to the original requires less effort to edit. However, there are cases where this relationship is not strong. For example, in the Decorator pattern, GPT-4 with design pattern knowledge had CS of 31.44% and ES of 35.37%, but without design pattern knowledge, CS increased and ES decreased. Model performance variability was evident, with GPT-4o and Llama-31-70B excelling in multiple patterns, while models like CodeQwen struggled.

Insights: Results reveal a nuanced relationship between CS and ES, indicating that high CS does not always equate to reduced editing effort for programmers. While some LLMs generate code resembling the original, these outputs may still require significant modifications due to underlying issues. It highlights the need for future Code LLM to prioritize better integration of design pattern knowledge and the generation of code that minimizes manual edits. Enhancing both accuracy and practical utility makes LLMs more reliable in software development, saving time and reducing error potential.

IV. CONCLUSION AND FUTURE WORK

This paper evaluates Code LLMs' ability to understand and generate code following design patterns. Experiments on classification, line completion, and function generation reveal that while design pattern knowledge often improves performance, some models rely on internalized patterns effectively. The study highlights challenges with complex patterns like Builder and nuances in the relationship between Code Similarity and Edit Similarity, emphasizing the need for improved design pattern integration and practical utility to enhance LLM reliability and reduce developer effort. We plan to expand the dataset to cover more object-oriented languages and incorporate more high-quality repositories. Additionally, we will observe evaluation results from more dimensions to ensure the reliability and practical guidance of our analyses.

REFERENCES

[1] Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., ..., et al. (2023). Gpt-4 technical report. arXiv preprint arXiv:2303.08774.

[2] Anthropic. Claude 3.5: Sonnet. Anthropic.com, 2024.

[3] Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., ... , Zhu, T. (2023). Qwen technical report. arXiv preprint arXiv:2309.16609.

[4] He, Z., Shu, W., Ge, X., Chen, L., Wang, J., Zhou, Y., ... , et al. (2024). Llama Scope: Extracting Millions of Features from Llama-3.1-8B with Sparse Autoencoders. arXiv preprint arXiv:2410.20526.

[5] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. arXiv preprint arXiv:2310.06825, 2023.

[6] Pan, Z., Luo, H., Li, M. and Liu, H., 2024. Chain-of-action: Faithful and multimodal question answering through large language models. arXiv preprint arXiv:2403.17359.

[7] Pan, Z., Luo, H., Li, M. and Liu, H., 2024. Conv-coa: Improving open-domain question answering in large language models via conversational chain-of-action. arXiv preprint arXiv:2405.17822.

[8] Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Heng Li, Jiangcheng Zhu, Jianqun Chen, Jing Chang, et al. Yi: Open foundation models by 01. ai. arXiv preprint arXiv:2403.04652, 2024.

[9] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2 technical report. arXiv preprint arXiv:2407.10671, 2024.

[10] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., et al. (2021). Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374.

[11] Chen, B., Zhang, F., Nguyen, A., Zan, D., Lin, Z., Lou, J. G., Chen, W. (2022). Codet: Code generation with generated tests. arXiv preprint arXiv:2207.10397.

[12] Siddiq, M. L., Majumder, S. H., Mim, M. R., Jajodia, S., Santos, J. C. (2022, October). An empirical study of code smells in transformer-based code generation techniques. In 2022 IEEE 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM) (pp. 71-82). IEEE.

[13] Yu, H., Shen, B., Ran, D., Zhang, J., Zhang, Q., Ma, Y., et al. (2024, February). Codereval: A benchmark of pragmatic code generation with generative pre-trained models. In Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (pp. 1-12).

[14] Hashtroudi, S., Shin, J., Hemmati, H., Wang, S. (2023). Automated test case generation using code models and domain adaptation. arXiv preprint arXiv:2308.08033.

[15] Lozhkov, A., Li, R., Allal, L. B., Cassano, F., Lamy-Poirier, J., Tazi, N., et al. (2024). Starcoder 2 and the stack v2: The next generation. arXiv preprint arXiv:2402.19173.

[16] Li, R., Allal, L. B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., et al. (2023). Starcoder: may the source be with you!. arXiv preprint arXiv:2305.06161.

[17] Pan, Z., Cao, R., Cao, Y., Ma, Y., Li, B., Huang, F., Liu, H. and Li, Y., 2024. Codev-Bench: How Do LLMs Understand Developer-Centric Code Completion?. arXiv preprint arXiv:2410.01353.